



PREMIER UNIVERSITY CHATTOGRAM

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT

COURSE NAME	Big Data Analysis Laboratory	
COURSE CODE	CSE 4346	
PROJECT TITLE	Steam Game Recommendations Data Analysis using Hadoop	
DATE OF REPORT	08-09-2026	
DATE OF SUBMISSION	15-09-2026	
SUBMITTED TO		
MD Tamim Hossain Lecturer Department of Computer Science and Engineering		
REMARKS	SUBMITTED BY	
	NAME	Rimjhim Dey
	ID	0222220005101039
	SEMESTER	8th
	BATCH	42nd
	SESSION	Spring 2026
	SECTION	A

Contents

1	Problem Statement	2
2	Dataset Description	3
3	Summary of Data Analysis	3
4	Environment and Loading the Data	3
5	Map Reduce Analysis	5
5.1	Review Summary per Game — Summarization	5
5.2	Binning Games by Rating — Data Organization	8
5.3	Games Joined with their Reviews — Reduce-side Join	10
6	Pig Analysis	13
6.1	Top 5 Rating Groups	13
6.2	Top 10 Most Reviewed Games	14
6.3	Top 10 Most Reviewed Games in each Rating Group	15
6.4	Top 10 Games by Total Hours Played	16
6.5	Top 10 Games by Total Hours Played in each Rating Group	18
7	Monitoring the Jobs	19
8	Verification	20
9	What the Data Says	20
10	Conclusion	21
	References	21

1. Problem Statement

Analyse the Steam game-recommendations dataset with Apache Hadoop (MapReduce) and Apache Pig, across its game and review tables, to find which games and rating groups draw the most reviews and the most hours of play on the Steam platform. The work follows the course brief:

- **Three MapReduce programs in Java**, each built on a different design pattern — summarization, data organization (binning) and a reduce-side join.
- **Five Apache Pig Latin scripts** for ranking and grouping questions on the same data, including a map-side replicated join over the full 41-million-row review table.

Everything was run on a single-node Hadoop cluster, and every job result in this report was read back out of HDFS after the job that produced it had finished.

2. Dataset Description

Dataset: Game Recommendations on Steam (Kaggle)

<https://www.kaggle.com/datasets/antonkozyriev/game-recommendations-on-steam>

A crawl of the Steam store and its user reviews: three related CSV tables and one JSON file of game descriptions. `app_id` links a review to its game and `user_id` to its author. The analyses use `games.csv` and `recommendations.csv`; `users.csv` is listed for completeness. All three CSV files are comma separated with a header row and no quoted fields, and a check before any job was written found every row with the expected number of columns (13, 3 and 8), so no cleaning step was needed beyond skipping the header line.

File	Size	Rows	One row is
<code>recommendations.csv</code>	2.0 GB	41,154,794	one user review of one game
<code>users.csv</code>	193 MB	14,306,064	one Steam user
<code>games.csv</code>	4.9 MB	50,872	one game on the Steam store
<code>games_metadata.json</code>	18 MB	50,872	description and tags of a game (not used)

Table 1: Files in the dataset (row counts exclude the header line).

games.csv				recommendations.csv			
#	Field	Type	Example / meaning	#	Field	Type	Meaning
0	<code>app_id</code>	int	13500 (key)	0	<code>app_id</code>	int	game → games
1	<code>title</code>	string	Prince of Persia: Warrior Within™	1	<code>helpful</code>	int	“helpful” votes
2	<code>date_release</code>	date	2008-11-21	2	<code>funny</code>	int	“funny” votes
3–5	<code>win, mac, linux</code>	boolean	supported platforms	3	<code>date</code>	date	review date
6	<code>rating</code>	string	Very Positive (Steam label, 9 values)	4	<code>is_recommended</code>	boolean	thumbs up / down
7	<code>positive_ratio</code>	int	84 (% positive reviews)	5	<code>hours</code>	float	hours played
8	<code>user_reviews</code>	int	2199 (reviews on Steam)	6	<code>user_id</code>	int	reviewer → users
9–10	<code>price_final/_original</code>	float	9.99 (USD)	7	<code>review_id</code>	int	review id
11	<code>discount</code>	float	0.0 (%)	users.csv: <code>user_id, products, reviews</code>			
12	<code>steam_deck</code>	boolean	runs on the Steam Deck				

Table 2: Schema of the three tables.

The rating column is the label Steam shows on a store page. It takes nine values and is the category field used for grouping and binning throughout, the role the video category played in the YouTube lab.

3. Summary of Data Analysis

MapReduce design patterns

1. Summarization — custom Writable and a combiner
2. Data Organization — binning with MultipleOutputs
3. Join Patterns — reduce-side join with MultipleInputs

MapReduce analyses

1. Review summary per game (*summarization*)
2. Binning games by rating (*data organization*)
3. Games joined with their reviews (*reduce-side join*)

Pig analyses

1. Top 5 rating groups by number of games
2. Top 10 most reviewed games
3. Top 10 most reviewed games in each rating group
4. Top 10 games by total hours played
5. Top 10 games by hours played in each rating group

4. Environment and Loading the Data

Component	Version / setting
Operating system	Pop!_OS Linux, 8 CPU cores, 16 GB RAM
Apache Hadoop HDFS	3.4.1, pseudo-distributed (single node), <code>fs.defaultFS = hdfs://localhost:9000</code> replication 1, block size 128 MB
Apache Pig	0.18.0, run in mapreduce mode on YARN
Java	OpenJDK 11.0.31
YARN memory	NodeManager 5120 MB; map 768 MB, reduce 1024 MB, AM 768 MB containers

Table 3: Software used.

The laptop had only about 2 GB of RAM free, and with Hadoop's default settings YARN would try to run eight 1 GB task containers at once, so the NodeManager was capped at 5 GB and the containers made smaller in `yarn-site.xml` and `mapred-site.xml`. The cluster is started daemon by daemon followed by the JobHistory server; the three CSV files are then copied into `/steam` in HDFS, where the 2 GB review file is stored as 16 blocks of 128 MB and `hdfs fsck` reports the directory healthy. Each job is compiled against the local Hadoop classpath into its own jar by `scripts/build.sh`.

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ jps
799676 ResourceManager
880252 JobHistoryServer
799581 SecondaryNameNode
799293 NameNode
880622 Jps
799401 DataNode
799792 NodeManager
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 1: All six Hadoop daemons running (jps).

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ hadoop version | head -1; pig -version 2>&1
| grep -i 'Apache Pig'; java -version 2>&1 | head -1
Hadoop 3.4.1
Apache Pig version 0.18.0 (r1926484)
openjdk version "11.0.31" 2026-04-21
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 2: Hadoop, Pig and Java versions.

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ hdfs dfs -mkdir -p /steam && cd dataset && time hdfs dfs -put -f games.csv users.csv r
ecommendations.csv /steam/
2026-09-11 20:15:27,306 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java clas
ses where applicable
2026-09-11 20:15:28,552 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java clas
ses where applicable

real    0m11.513s
user    0m8.256s
sys     0m4.015s
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 3: Copying the dataset into HDFS (11.5 seconds).

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ hdfs dfs -ls -h /steam && hdfs fsck /steam -files -blocks | grep -E 'recommendations.c
sv|Total blocks|Status'
2026-09-11 20:15:40,134 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java clas
ses where applicable
Found 3 items
-rw-r--r-- 1 rii supergroup 4.6 M 2026-09-11 20:15 /steam/games.csv
-rw-r--r-- 1 rii supergroup 1.9 G 2026-09-11 20:15 /steam/recommendations.csv
-rw-r--r-- 1 rii supergroup 184.1 M 2026-09-11 20:15 /steam/users.csv
2026-09-11 20:15:41,784 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java clas
ses where applicable
Connecting to namenode via http://localhost:9870/fsck?ugi=rii&files=1&blocks=1&path=%2Fsteam
/steam/recommendations.csv 2023223067 bytes, replicated: replication=1, 16 block(s): OK
Status: HEALTHY
Total blocks (validated): 19 (avg. block size 116902888 B)
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 4: The dataset in HDFS: 1.9 GB of reviews stored as 16 blocks.

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ ./scripts/build.sh && ls -l */*.jar
built GameReviewSummary/GameReviewSummary.jar
built BinningByRating/BinningByRating.jar
built GameReviewJoin/GameReviewJoin.jar
-rw-rw-r-- 1 rii rii 2849 Sep 11 20:18 BinningByRating/BinningByRating.jar
-rw-rw-r-- 1 rii rii 6076 Sep 11 20:18 GameReviewJoin/GameReviewJoin.jar
-rw-rw-r-- 1 rii rii 4996 Sep 11 20:18 GameReviewSummary/GameReviewSummary.jar
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 5: The three jars built.

Hadoop

Overview

Datanodes

Datanode Volume Failures

Snapshot

Startup Progress

Utilities

Overview 'localhost:9000' (-active)

Started:	Fri Sep 11 20:14:05 +0800 2026
Version:	3.4.1 - h4076703054895613840b0a03220784228461
Compiled:	Wed Oct 09 20:57:00 +0800 2024 by mthaku from branch-3.4.1
Cluster ID:	C1D-0595238-cd8f-4b3c-4b4f-R6d433d7
Block Pool ID:	BP-1496064727-100.125.247.109-176389801002

Summary

Security is off.
SafeMode is off.

174 files and directories, 138 blocks (108 replicated blocks, 0 erasure coded block groups) = 262 total filesystem object(s).
Heap Memory used 232.02 MB of 256 MB heap memory. Max heap memory is 3.84 GB.
Non-Heap Memory used 62.83 MB of 62.56 MB Committed Non-Heap Memory. Max non-heap memory is non-bounded.

Configured Capacity:	234.70 GB
Configured Remote Capacity:	0 B
DFS Used:	2.1 GB (0.94%)
Non DFS Used:	93.59 GB
DFS Remaining:	117.61 GB (52.20%)
Block Pool Used:	2.1 GB (0.94%)
Datanodes usage% (Min/Max/Used/Dev):	0.94% / 0.94% / 0.94% / 0.00%
Live Nodes:	1 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes:	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes:	0
Entering Maintenance Nodes:	0

Figure 6: NameNode overview (localhost:9870).

About the screenshots. To keep the report short, each job run is shown as one screenshot of the key lines of its console output — the command, job id, progress, completion, the counters that matter and the running time — with the skipped lines marked [... n lines ...]. The complete console output of every command is in the logs/ folder of the repository, and full-length screenshots are in screenshots/. Import statements are left out of the code listings; the line numbers are those of the source files.

5. Map Reduce Analysis

5.1 Review Summary per Game — Summarization

Returns, for every game, how many reviews it received, how many recommend it, how long its reviewers played it and how many helpful votes its reviews collected (GameReviewSummary).

This is the **numerical summarization** pattern. The mapper emits the game's app_id with a small tuple — a count of one, a 1 or 0 for “recommended”, the hours played and the helpful votes. Every field is additive (a count, a sum or a maximum), so partial results can be merged in any order; that is what makes a **combiner** valid, and the reducer finishes with exactly the same code. The analysis consists of a **Driver** (registers the reducer as the combiner too), a **Mapper** (emits app_id and a ReviewStatsTuple), a **Reducer** (merges all tuples of one game; also the combiner) and a **Tuple** class (the custom Writable).

Driver Class

```

1 package steam.summary;
2 /**
3  * MR-1 Review summary per game (numerical summarization
4  * pattern).
5  *
6  * For every app_id in recommendations.csv: number of reviews,
7  * number
8  * recommended, positive %, total / average / maximum hours
9  * played and total
10 * helpful votes.
11 *
12 * Usage: hadoop jar GameReviewSummary.jar steam.summary.
13 * GameReviewSummary <in> <out>
14 */
15 public class GameReviewSummary {
16
17     public static void main(String[] args) throws Exception {
18         if (args.length != 2) {
19             System.err.println("Usage: GameReviewSummary <input>
20 <output>");
21             System.exit(2);
22         }
23         Configuration conf = new Configuration();
24         Job job = Job.getInstance(conf, "Steam review summary
25 per game");
26         job.setJarByClass(GameReviewSummary.class);
27
28         job.setMapperClass(ReviewMapper.class);
29         job.setCombinerClass(ReviewReducer.class);
30         job.setReducerClass(ReviewReducer.class);
31
32         job.setOutputKeyClass(Text.class);
33         job.setOutputValueClass(ReviewStatsTuple.class);
34
35         FileInputFormat.addInputPath(job, new Path(args[0]));
36         FileOutputFormat.setOutputPath(job, new Path(args[1]));
37         System.exit(job.waitForCompletion(true) ? 0 : 1);
38     }
39 }

```

Mapper Class

```

1 package steam.summary;
2 /**
3  * Reads recommendations.csv
4  * (app_id, helpful, funny, date, is_recommended, hours, user_id
5  * , review_id)
6  * and emits (app_id, tuple) for every review, with a count of
7  * one.
8  */
9 public class ReviewMapper extends Mapper<LongWritable, Text,
10 Text, ReviewStatsTuple> {
11
12     private final Text appId = new Text();
13     private final ReviewStatsTuple tuple = new ReviewStatsTuple
14 ();
15
16     @Override
17     protected void map(LongWritable key, Text value, Context
18 context)
19         throws IOException, InterruptedException {
20         String[] f = value.toString().split(",");
21         if (f.length != 8 || f[0].equals("app_id")) {
22             return; // header or malformed row
23         }
24         try {
25             long helpful = Long.parseLong(f[1]);
26             boolean recommended = Boolean.parseBoolean(f[4]);
27             double hours = Double.parseDouble(f[5]);
28             appId.set(f[0]);
29             tuple.set(1, recommended ? 1 : 0, hours, hours,
30 helpful);
31             context.write(appId, tuple);
32         } catch (NumberFormatException e) {
33             context.getCounter("Summary", "Malformed rows").
34 increment(1);
35         }
36     }
37 }

```

Reducer Class (also the Combiner)

```

1 package steam.summary;
2 /**
3  * Merges every tuple for one app_id into a single summary.
4  * Input and output types match, so the driver also registers
5  * this class as
6  * the combiner: each mapper pre-merges its own tuples before
7  * the shuffle.
8  */
9 public class ReviewReducer extends Reducer<Text,
10     ReviewStatsTuple, Text, ReviewStatsTuple> {
11
12     private final ReviewStatsTuple result = new ReviewStatsTuple
13         ();
14
15     @Override
16     protected void reduce(Text key, Iterable<ReviewStatsTuple>
17         values, Context context)
18         throws IOException, InterruptedException {
19         result.set(0, 0, 0, 0, 0);
20         for (ReviewStatsTuple t : values) {
21             result.merge(t);
22         }
23         context.write(key, result);
24     }
25 }

```

Tuple Class

```

1 package steam.summary;
2 /**
3  * Custom Writable that carries the running totals for one game.
4  * Every field is additive (count, sum, max), so the same tuple
5  * can be merged
6  * in the combiner and again in the reducer without losing
7  * accuracy.
8  */
9 public class ReviewStatsTuple implements Writable {
10
11     private long reviews;        // number of recommendations
12     private long recommended;    // how many of them were thumbs
13     // up
14     private double totalHours;    // hours played, summed over
15     // reviewers
16     private double maxHours;      // most hours any single
17     // reviewer played
18     private long helpful;         // "helpful" votes, summed
19
20     public ReviewStatsTuple() {
21     }
22 }

```

```

17
18 public void set(long reviews, long recommended, double
19     totalHours, double maxHours, long helpful) {
20     this.reviews = reviews;
21     this.recommended = recommended;
22     this.totalHours = totalHours;
23     this.maxHours = maxHours;
24     this.helpful = helpful;
25 }
26
27 /** Folds another partial result into this one. */
28 public void merge(ReviewStatsTuple other) {
29     reviews += other.reviews;
30     recommended += other.recommended;
31     totalHours += other.totalHours;
32     maxHours = Math.max(maxHours, other.maxHours);
33     helpful += other.helpful;
34 }
35
36 public void write(DataOutput out) throws IOException {
37     out.writeLong(reviews);
38     out.writeLong(recommended);
39     out.writeDouble(totalHours);
40     out.writeDouble(maxHours);
41     out.writeLong(helpful);
42 }
43
44 public void readFields(DataInput in) throws IOException {
45     reviews = in.readLong();
46     recommended = in.readLong();
47     totalHours = in.readDouble();
48     maxHours = in.readDouble();
49     helpful = in.readLong();
50 }
51
52 /** reviews, recommended, positive %, total hours, average
53     hours, max hours, helpful votes */
54 @Override
55 public String toString() {
56     double positivePct = reviews == 0 ? 0 : 100.0 *
57         recommended / reviews;
58     double avgHours = reviews == 0 ? 0 : totalHours /
59         reviews;
60     return reviews + "\t" + recommended + "\t" + String.
61         format("%.1f", positivePct) + "\t"
62         + String.format("%.1f", totalHours) + "\t" +
63         String.format("%.1f", avgHours) + "\t"
64         + String.format("%.1f", maxHours) + "\t" +
65         helpful;
66 }

```

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ time hadoop jar GameReviewSummary/GameReviewSummary.jar steam.summary.GameReviewSummary /steam/recommendations.csv /steam/out_summary
[... 5 lines ...]
2026-09-11 20:18:46,584 INFO mapreduce.JobSubmitter: number of splits:15
[... 6 lines ...]
2026-09-11 20:18:47,743 INFO mapreduce.Job: Running job: job_1789136053647_0002
[... 1 line ...]
2026-09-11 20:18:54,916 INFO mapreduce.Job: map 0% reduce 0%
[... 32 lines ...]
2026-09-11 20:20:57,896 INFO mapreduce.Job: map 100% reduce 100%
2026-09-11 20:20:58,913 INFO mapreduce.Job: Job job_1789136053647_0002 completed successfully
[... 14 lines ...]
    Killed map tasks=1
    Launched map tasks=16
    Launched reduce tasks=1
[... 10 lines ...]
    Map input records=41154795
    Map output records=41154794
[... 3 lines ...]
    Combine input records=41309286
    Combine output records=245258
[... 2 lines ...]
    Reduce input records=90766
    Reduce output records=37610
[... 25 lines ...]
real    2m15.426s
[... 2 lines ...]
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 7: Execution: the review summary job over the 2 GB review file (key lines of the console output).

The job read all 41,154,794 reviews in 15 input splits and finished in 2 minutes 15 seconds. The combiner at work is visible in the counters: the mappers produced 41,154,794 records, but only 90,766 reached the reducer — roughly 450 times less data through the shuffle. The reducer wrote one line for each of the **37,610 games** that have at least one review.

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ hdfs dfs -ls /steam/out_summary && hdfs dfs -cat /steam/out_summary/part-r-00000 | sort -t'$'\t' -k2 -nr | head -10
2026-09-11 20:21:28,716 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 2 items
-rw-r--r-- 1 rii supergroup 0 2026-09-11 20:20 /steam/out_summary/_SUCCESS
-rw-r--r-- 1 rii supergroup 1441145 2026-09-11 20:20 /steam/out_summary/part-r-00000
2026-09-11 20:21:29,853 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
440 319492 294879 92.3 101809977.6 318.7 999.9 521978
252490 270684 226293 83.6 92944004.6 343.4 999.9 771227
1091500 226414 168631 74.5 23146491.9 102.2 999.1 1189869
730 219737 186306 84.8 94260307.3 429.0 999.9 1910071
570 216914 171841 79.2 93072785.4 429.1 999.9 842346
444090 204176 171516 84.0 28549772.6 139.8 999.9 188983
292030 204166 193139 94.6 31522739.6 154.4 999.9 813793
377160 196789 160497 81.6 40149411.3 204.0 999.9 899972
374320 190157 176742 92.9 32467657.9 170.7 999.9 529674
431960 190129 186792 98.2 19547974.7 102.8 999.8 802753
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 8: Output: the ten games with the most reviews. Columns: app_id, reviews, recommended, positive %, total hours, average hours, maximum hours, helpful votes.

app_id	Game	Reviews	Positive %	Avg. hours	Helpful votes
440	Team Fortress 2	319,492	92.3	318.7	521,978
252490	Rust	270,684	83.6	343.4	771,227
1091500	Cyberpunk 2077	226,414	74.5	102.2	1,189,869
730	Counter-Strike: Global Offensive	219,737	84.8	429.0	1,910,071
570	Dota 2	216,914	79.2	429.1	842,346
444090	Paladins®	204,176	84.0	139.8	188,983
292030	The Witcher® 3: Wild Hunt	204,166	94.6	154.4	813,793
377160	Fallout 4	196,789	81.6	204.0	899,972
374320	DARK SOULS™ III	190,157	92.9	170.7	529,674
431960	Wallpaper Engine	190,129	98.2	102.8	802,753

Table 4: The same ten games with their titles looked up from games.csv.

The maximum-hours column reads 999.9 for almost every popular game. That is not a bug in the job: the hours

field in the source data never goes above 1,000, so for any game with a large player base at least one reviewer sits at the cap.

5.2 Binning Games by Rating — Data Organization

Performs binning on the Steam rating label, writing every game into one output file per bin. The program is BinningByRating.

This is the **binning** pattern. The job is map-only: each mapper decides which bin a record belongs to and writes it straight to that bin's file through MultipleOutputs — no shuffle, no sort, no reducer. The bin name is the rating with spaces removed ("Very Positive" becomes VeryPositive-m-00000), a counter per bin records how many games landed in it, and LazyOutputFormat stops Hadoop writing an empty default part-m-00000. The analysis consists of a **Driver** (zero reducers, registers the bins named output) and a **Mapper** (writes each game to its bin and counts it).

Driver Class

```
1 package steam.binning;
2 /**
3  * MR-2 Binning games by rating (data organization pattern:
4  *   binning).
5  *
6  * Map-only job. Every game is written to one of nine output
7  *   files, one per
8  *   Steam rating label, using MultipleOutputs. No reducer and no
9  *   shuffle.
10  *
11  * Usage: hadoop jar BinningByRating.jar steam.binning.
12  *   BinningByRating <in> <out>
13  */
14 public class BinningByRating {
15
16     public static void main(String[] args) throws Exception {
17         if (args.length != 2) {
18             System.err.println("Usage: BinningByRating <input> <
19             output>");
20             System.exit(2);
21         }
22         Configuration conf = new Configuration();
23         Job job = Job.getInstance(conf, "Steam binning games by
24         rating");
25         job.setJarByClass(BinningByRating.class);
26
27         job.setMapperClass(BinningMapper.class);
28         job.setNumReduceTasks(0);
29
30         job.setOutputKeyClass(Text.class);
31         job.setOutputValueClass(NullWritable.class);
32
33         MultipleOutputs.addNamedOutput(job, "bins",
34         TextOutputFormat.class, Text.class, NullWritable.class);
35         // only create the bin files; skip the empty default
36         part-m-00000
37         LazyOutputFormat.setOutputFormatClass(job,
38         TextOutputFormat.class);
39
40         FileInputFormat.addInputPath(job, new Path(args[0]));
41         FileOutputFormat.setOutputPath(job, new Path(args[1]));
42         System.exit(job.waitForCompletion(true) ? 0 : 1);
43     }
44 }
```

Mapper Class

```
1 package steam.binning;
2 /**
3  * Reads games.csv and writes each game into the bin (output
4  *   file) named after
5  *   its Steam rating label, e.g. "Very Positive" -> VeryPositive-
6  *   m-00000.
7  *
8  * A counter per bin records how many games landed in it.
9  */
10 public class BinningMapper extends Mapper<LongWritable, Text,
11     Text, NullWritable> {
12
13     private MultipleOutputs<Text, NullWritable> mos;
14
15     @Override
16     protected void setup(Context context) {
17         mos = new MultipleOutputs<>(context);
18     }
19
20     @Override
21     protected void map(LongWritable key, Text value, Context
22     context)
23     throws IOException, InterruptedException {
24         // app_id,title,date_release,win,mac,linux,rating,
25         positive_ratio,user_reviews,
26         // price_final,price_original,discount,steam_deck
27         String[] f = value.toString().split(",");
28         if (f.length != 13 || f[0].equals("app_id")) {
29             return; // header or malformed row
30         }
31         String rating = f[6].trim();
32         String bin = rating.replaceAll("[^A-Za-z]", ""); // file
33         names must be alphanumeric
34         mos.write("bins", value, NullWritable.get(), bin);
35         context.getCounter("Bins", rating).increment(1);
36     }
37
38     @Override
39     protected void cleanup(Context context) throws IOException,
40     InterruptedException {
41         mos.close();
42     }
43 }
```

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ hadoop jar BinningByRating/BinningByRating.jar steam.binning.BinningByRating /steam/games.csv /steam/out_binning
[... 5 lines ...]
2026-09-11 20:18:20,298 INFO mapreduce.JobSubmitter: number of splits:1
[... 6 lines ...]
2026-09-11 20:18:21,444 INFO mapreduce.Job: Running job: job_1789136053647_0001
[... 1 line ...]
2026-09-11 20:18:29,551 INFO mapreduce.Job: map 0% reduce 0%
2026-09-11 20:18:34,624 INFO mapreduce.Job: map 100% reduce 0%
2026-09-11 20:18:35,641 INFO mapreduce.Job: Job job_1789136053647_0001 completed successfully
[... 14 lines ...]
    Launched map tasks=1
[... 7 lines ...]
    Map input records=50873
    Map output records=0
[... 11 lines ...]
    Bins
        Mixed=12157
        Mostly Negative=1849
        Mostly Positive=8738
        Negative=303
        Overwhelmingly Negative=14
        Overwhelmingly Positive=1110
        Positive=13502
        Very Negative=60
        Very Positive=13139
[... 4 lines ...]
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 9: Execution: the binning job, with the nine Bins counters (key lines of the console output).

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ hdfs dfs -ls /steam/out_binning && hdfs dfs -cat /steam/out_binning/OverwhelminglyNegative-m-00000
2026-09-11 20:18:45,564 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 10 items
-rw-r--r-- 1 rii supergroup 1090434 2026-09-11 20:18 /steam/out_binning/Mixed-m-00000
-rw-r--r-- 1 rii supergroup 185177 2026-09-11 20:18 /steam/out_binning/MostlyNegative-m-00000
-rw-r--r-- 1 rii supergroup 872438 2026-09-11 20:18 /steam/out_binning/MostlyPositive-m-00000
-rw-r--r-- 1 rii supergroup 28067 2026-09-11 20:18 /steam/out_binning/Negative-m-00000
-rw-r--r-- 1 rii supergroup 1570 2026-09-11 20:18 /steam/out_binning/OverwhelminglyNegative-m-00000
-rw-r--r-- 1 rii supergroup 119286 2026-09-11 20:18 /steam/out_binning/OverwhelminglyPositive-m-00000
-rw-r--r-- 1 rii supergroup 1273597 2026-09-11 20:18 /steam/out_binning/Positive-m-00000
-rw-r--r-- 1 rii supergroup 6409 2026-09-11 20:18 /steam/out_binning/VeryNegative-m-00000
-rw-r--r-- 1 rii supergroup 1286690 2026-09-11 20:18 /steam/out_binning/VeryPositive-m-00000
-rw-r--r-- 1 rii supergroup 0 2026-09-11 20:18 /steam/out_binning/_SUCCESS
2026-09-11 20:18:47,024 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
1180320,War of the Three Kingdoms,2021-12-17,true,false,false,Overwhelmingly Negative,15,21276,0.0,0.0,0.0,true
1416420,Expansion - Europa Universalis IV: Leviathan,2021-04-27,true,true,true,Overwhelmingly Negative,10,4723,19.99,19.99,0.0,true
512032,Sid Meier's Civilization® VI: Vikings Scenario Pack,2016-12-20,true,true,true,Overwhelmingly Negative,15,1184,0.0,0.0,0.0,true
2357570,Overwatch® 2,2023-08-10,true,false,false,Overwhelmingly Negative,9,181198,0.0,0.0,0.0,true
1850380,Tricolour Lovestory TrueEnd,2022-01-27,true,false,false,Overwhelmingly Negative,14,1739,0.99,0.99,0.0,true
2016940,02Jam Online,2023-01-16,true,false,false,Overwhelmingly Negative,17,1689,0.0,0.0,0.0,true
1799220,Malice,2022-11-02,true,false,false,Overwhelmingly Negative,16,723,7.99,7.99,0.0,true
267600,Airport Simulator 2014,2013-12-19,true,false,false,Overwhelmingly Negative,13,553,4.99,4.99,0.0,true
1563130,SuperPower 3,2022-10-07,true,false,false,Overwhelmingly Negative,10,880,29.99,29.99,0.0,true
201510,Flatout 3: Chaos & Destruction,2011-12-13,true,false,false,Overwhelmingly Negative,15,2766,9.99,9.99,0.0,true
1162602,TEKKEN 7 - DLC13: Frame Data Display,2019-12-10,true,false,false,Overwhelmingly Negative,14,605,1.49,2.99,50.0,true
246090,Spacebase DF-9,2014-10-27,true,true,true,Overwhelmingly Negative,18,3261,9.99,9.99,0.0,true
573831,Tom Clancy's Ghost Recon® Wildlands - Narco Road,2017-04-25,true,false,false,Overwhelmingly Negative,16,564,14.99,14.99,0.0,true
227160,Kinetic Void,2014-11-21,true,true,true,Overwhelmingly Negative,17,1381,19.99,19.99,0.0,true
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 10: Output: the nine bin files in HDFS, and the smallest bin (Overwhelmingly Negative, 14 games) printed in full.

One map task read the 50,873 lines of `games.csv` (50,872 games and the header). Map output records is 0 because `MultipleOutputs` writes to the bin files directly; the Bins counters are the real result, and the nine of them add up to 50,872 — every game placed in exactly one bin.

Bin (rating)	Games	Share	Bin (rating)	Games	Share
Positive	13,502	26.5%	Overwhelmingly Positive	1,110	2.2%
Very Positive	13,139	25.8%	Negative	303	0.6%
Mixed	12,157	23.9%	Very Negative	60	0.1%
Mostly Positive	8,738	17.2%	Overwhelmingly Negative	14	0.03%
Mostly Negative	1,849	3.6%	Total	50,872	100%

Table 5: Games per bin, from the job's Bins counters.

5.3 Games Joined with their Reviews — Reduce-side Join

Returns each game's title and rating together with how many people reviewed it and how many hours they played it (GameReviewJoin).

This is a **reduce-side join**. The two inputs have different layouts, so each gets its own mapper through `MultipleInputs`; both key on `app_id` and tag their values — G for a game, R for review data — so the reducer, which receives everything for one `app_id` together, can put the two sides back together. Sending 41 million reviews through the shuffle one by one would be slow, so the review mapper uses **in-mapper combining**: a running total per game in a hash map, emitted once per game when its split is done. The join is inner — games nobody reviewed, and reviews whose game is missing, are dropped and counted. The analysis consists of a **Driver**, a **Game Mapper**, a **Recommendation Mapper** and a **Join Reducer**.

Driver Class

```

1 package steam.join;
2 /**
3  * MR-3 Games joined with their reviews (join pattern: reduce-
4   * side join).
5  *
6  * *
7  * Two inputs, each with its own mapper (MultipleInputs), meet
8   * in the reducer
9  *
10  * on app_id. The result puts each game's title and rating next
11   * to how many
12  *
13  * people reviewed it and how many hours they played.
14  *
15  * *
16  * Usage: hadoop jar GameReviewJoin.jar steam.join.
17   *
18   * GameReviewJoin <games> <recommendations> <out>
19  */
20
21 public class GameReviewJoin {
22
23     public static void main(String[] args) throws Exception {
24         if (args.length != 3) {
25             System.err.println("Usage: GameReviewJoin <games> <
26 recommendations> <output>");
27             System.exit(2);
28         }
29
30         Configuration conf = new Configuration();
31         Job job = Job.getInstance(conf, "Steam games joined with
32 reviews");
33         job.setJarByClass(GameReviewJoin.class);
34
35         MultipleInputs.addInputPath(job, new Path(args[0]),
36 TextInputFormat.class, GameMapper.class);
37         MultipleInputs.addInputPath(job, new Path(args[1]),
38 TextInputFormat.class, RecommendationMapper.class);
39         job.setReducerClass(JoinReducer.class);
40
41         job.setOutputKeyClass(Text.class);
42         job.setOutputValueClass(Text.class);
43
44         FileOutputFormat.setOutputPath(job, new Path(args[2]));
45         System.exit(job.waitForCompletion(true) ? 0 : 1);
46     }
47 }

```

Game Mapper

```

1 package steam.join;
2 /**
3  * Left side of the join: games.csv.
4  * Emits (app_id, "G<TAB>title<TAB>rating") so the reducer can
      tell a game
5  * record apart from review records arriving under the same key.
6  */
7 public class GameMapper extends Mapper<LongWritable, Text, Text,
      Text> {
8
9     private final Text appId = new Text();
10    private final Text tagged = new Text();
11
12    @Override
13    protected void map(LongWritable key, Text value, Context
      context)
14        throws IOException, InterruptedException {
15        String[] f = value.toString().split(",");
16        if (f.length != 13 || f[0].equals("app_id")) {
17            return; // header or malformed row
18        }
19        appId.set(f[0]);
20        tagged.set("G\t" + f[1] + "\t" + f[6]);
21        context.write(appId, tagged);
22    }
23 }

```

Recommendation Mapper

```

1 package steam.join;
2 /**
3  * Right side of the join: recommendations.csv (41 million rows)
4  *
5  * Emitting one record per review would push 41 million records
6  * through the
7  * shuffle. Instead the mapper keeps a running total per app_id
8  * in memory
9  * (in-mapper combining) and emits one "R<TAB>reviews<TAB>
10  * recommended<TAB>hours"
11  * record per game when its input split is finished.
12  */
13 public class RecommendationMapper extends Mapper<LongWritable,
14     Text, Text, Text> {
15
16     // app_id -> {reviews, recommended, hours}
17     private final Map<String, double[]> totals = new HashMap<>()
18     ;
19
20     @Override
21     protected void map(LongWritable key, Text value, Context
22         context) {
23         // app_id,helpful,funny,date,is_recommended,hours,
24         user_id,review_id
25         String[] f = value.toString().split(",");
26         if (f.length != 8 || f[0].equals("app_id")) {
27             return; // header or malformed row
28         }
29         try {

```

```

23         double hours = Double.parseDouble(f[5]);
24         double[] t = totals.computeIfAbsent(f[0], k -> new
double[3]);
25         t[0] += 1;
26         t[1] += Boolean.parseBoolean(f[4]) ? 1 : 0;
27         t[2] += hours;
28     } catch (NumberFormatException e) {
29         context.getCounter("Join", "Malformed review rows").
increment(1);
30     }
31 }
32
33 @Override
34 protected void cleanup(Context context) throws IOException,
InterruptedException {
35     Text appId = new Text();
36     Text tagged = new Text();
37     for (Map.Entry<String, double[]> e : totals.entrySet())
38     {
39         double[] t = e.getValue();
40         appId.set(e.getKey());
41         tagged.set("R\t" + (long) t[0] + "\t" + (long) t[1]
+ "\t" + t[2]);
42         context.write(appId, tagged);
43     }
44 }

```

Join Reducer

```

1 package steam.join;
2 /**
3  * Inner join on app_id. For each game the reducer receives one
4  * "G" record
5  * (title, rating) and several "R" records (partial review
6  * totals from each
7  * mapper). It adds up the R records and writes one joined line:
8  *
9  * app_id title rating reviews positive% total_hours
10  * avg_hours
11  *
12  * Games with no reviews, and reviews whose game is missing from
13  * games.csv,
14  * are dropped and counted.
15  */
16 public class JoinReducer extends Reducer<Text, Text, Text, Text>
{

```

```

13 private final Text out = new Text();
14
15
16 @Override
17 protected void reduce(Text key, Iterable<Text> values,
Context context)
18     throws IOException, InterruptedException {
19     String title = null;
20     String rating = null;
21     long reviews = 0;
22     long recommended = 0;
23     double hours = 0;
24
25     for (Text v : values) {
26         String[] f = v.toString().split("\t");
27         if (f[0].equals("G")) {
28             title = f[1];
29             rating = f[2];
30         } else {
31             reviews += Long.parseLong(f[1]);
32             recommended += Long.parseLong(f[2]);
33             hours += Double.parseDouble(f[3]);
34         }
35     }
36
37     if (title == null) {
38         context.getCounter("Join", "Reviews with no matching
game").increment(reviews);
39         return;
40     }
41     if (reviews == 0) {
42         context.getCounter("Join", "Games with no reviews").
increment(1);
43         return;
44     }
45     context.getCounter("Join", "Joined games").increment(1);
46     out.set(title + "\t" + rating + "\t" + reviews + "\t"
+ String.format("%.1f", 100.0 * recommended /
47 reviews) + "\t"
48 + String.format("%.1f", hours) + "\t"
49 + String.format("%.1f", hours / reviews));
50     context.write(key, out);
51 }
52 }

```

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ time hadoop jar GameReviewJoin/GameReviewJoin.jar steam.join.GameReviewJoin /steam/games.csv /steam/recommendations.csv /steam/out_join
[... 6 lines ...]
2026-09-11 20:21:28,549 INFO mapreduce.JobSubmitter: number of splits:16
[... 6 lines ...]
2026-09-11 20:21:29,501 INFO mapreduce.Job: Running job: job_1789136053647_0003
[... 1 line ...]
2026-09-11 20:21:34,598 INFO mapreduce.Job: map 0% reduce 0%
[... 10 lines ...]
2026-09-11 20:22:40,060 INFO mapreduce.Job: map 100% reduce 100%
2026-09-11 20:22:41,078 INFO mapreduce.Job: Job job_1789136053647_0003 completed successfully
[... 14 lines ...]
    Launched map tasks=16
    Launched reduce tasks=1
[... 10 lines ...]
    Map input records=41205668
    Map output records=141638
[... 3 lines ...]
    Combine input records=0
    Combine output records=0
[... 2 lines ...]
    Reduce input records=141638
    Reduce output records=37610
[... 13 lines ...]
    Join
        Games with no reviews=13262
        Joined games=37610
[... 12 lines ...]
real    1m15.075s
[... 2 lines ...]
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 11: Execution: the join job over both inputs, with the Join counters (key lines of the console output).

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ hdfs dfs -ls /steam/out_join && hdfs dfs -cat /steam/out_join/part-r-00000 | sort -t '$'\t' -k6 -gr | head -10
2026-09-11 20:23:22,534 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 2 items
-rw-r--r-- 1 rii supergroup 0 2026-09-11 20:22 /steam/out_join/_SUCCESS
-rw-r--r-- 1 rii supergroup 2155580 2026-09-11 20:22 /steam/out_join/part-r-00000
2026-09-11 20:23:23,525 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
440 Team Fortress 2 Very Positive 319492 92.3 101809977.6 318.7
730 Counter-Strike: Global Offensive Very Positive 219737 84.8 94260307.3 429.0
570 Dota 2 Very Positive 216914 79.2 93072785.4 429.1
252490 Rust Very Positive 270684 83.6 92944004.6 343.4
359550 Tom Clancy's Rainbow Six® Siege Very Positive 189603 81.9 68445823.3 361.0
1085660 Destiny 2 Very Positive 185527 81.9 62504172.8 336.9
394360 Hearts of Iron IV Very Positive 143187 91.3 53086631.1 370.8
346110 ARK: Survival Evolved Very Positive 159736 79.6 46720359.4 292.5
230410 Warframe Very Positive 127812 88.0 46450175.3 363.4
271590 Grand Theft Auto V Very Positive 163909 68.3 46170394.5 281.7
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 12: Output: the ten games with the most hours played. Columns: app_id, title, rating, reviews, positive %, total hours, average hours.

The job read 16 splits (15 from the review file, one from games.csv) and finished in 1 minute 15 seconds — faster than the summary job over the same 2 GB, because in-mapper combining cut the map output from 41 million records to 141,638. The Join counters show 37,610 games matched and 13,262 games with no reviews; no review pointed to a game missing from games.csv.

Game	Rating	Reviews	Total hours	Avg. hours
Team Fortress 2	Very Positive	319,492	101,809,977.6	318.7
Counter-Strike: Global Offensive	Very Positive	219,737	94,260,307.3	429.0
Dota 2	Very Positive	216,914	93,072,785.4	429.1
Rust	Very Positive	270,684	92,944,004.6	343.4
Tom Clancy's Rainbow Six® Siege	Very Positive	189,603	68,445,823.3	361.0
Destiny 2	Very Positive	185,527	62,504,172.8	336.9
Hearts of Iron IV	Very Positive	143,187	53,086,631.1	370.8
ARK: Survival Evolved	Very Positive	159,736	46,720,359.4	292.5
Warframe	Very Positive	127,812	46,450,175.3	363.4
Grand Theft Auto V	Very Positive	163,909	46,170,394.5	281.7

Table 6: Top 10 games by total hours played, from the joined output.

6. Pig Analysis

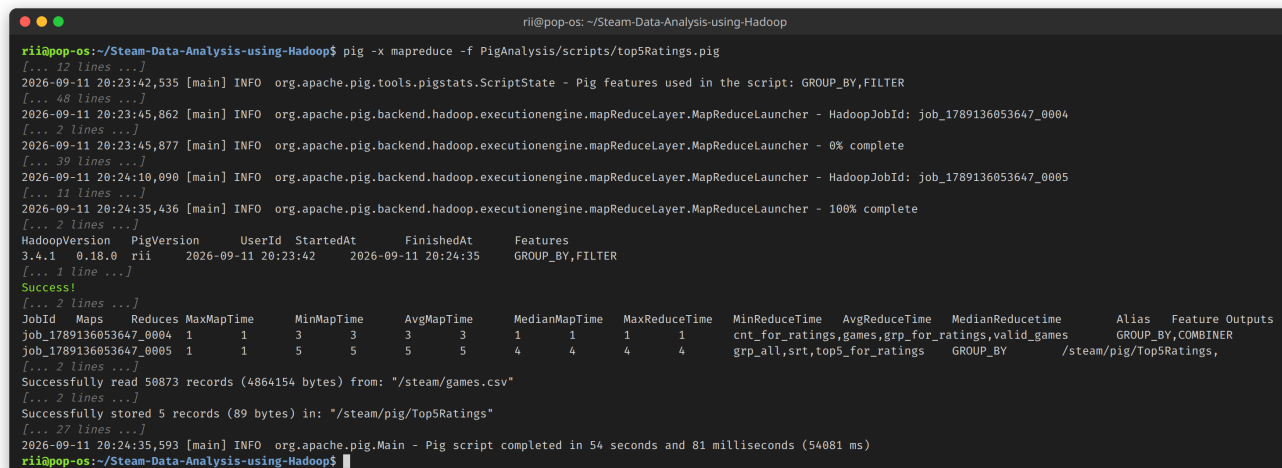
All five scripts read the dataset from HDFS with `PigStorage(',')` and drop the header row with `FILTER ... BY app_id IS NOT NULL` (the header's "app_id" fails the cast to int and becomes null — which is what the `FIELD_DISCARDED_TYPE_CONVERSION_FAILED` warning at the end of each run counts). Results are stored tab separated under `/steam/pig/`; a tab rather than a pipe, because 104 game titles contain a `|`. Every ranking sorts inside a nested `FOREACH` (`ORDER` then `LIMIT`) rather than with a top-level `ORDER BY`: for the "in each rating group" queries that is what the question needs, and for the overall rankings it avoids the sampling job a top-level `ORDER BY` starts.

6.1 Top 5 Rating Groups

Returns the five Steam rating labels that the most games carry. Games are grouped by rating and counted; the counts are gathered into one group (`GROUP ... ALL`) so they can be sorted together and cut to five. Pig compiles this into two MapReduce jobs — the count (with a combiner) and the ranking.

Script

```
1 -- Pig-1 Top 5 rating groups by number of games
2 games = LOAD '/steam/games.csv' USING PigStorage(',') AS
3   (app_id:int, title:chararray, date_release:chararray, win:chararray, mac:chararray,
4     linux:chararray, rating:chararray, positive_ratio:int, user_reviews:int,
5     price_final:double, price_original:double, discount:double, steam_deck:chararray);
6 valid_games = FILTER games BY app_id IS NOT NULL; -- drops the header row
7
8 grp_for_ratings = GROUP valid_games BY rating;
9 cnt_for_ratings = FOREACH grp_for_ratings GENERATE group AS rating, COUNT(valid_games) AS num_games;
10
11 grp_all = GROUP cnt_for_ratings ALL;
12 top5_for_ratings = FOREACH grp_all {
13   srt = ORDER cnt_for_ratings BY num_games DESC;
14   lim = LIMIT srt 5;
15   GENERATE FLATTEN(lim);
16 };
17 STORE top5_for_ratings INTO '/steam/pig/Top5Ratings' USING PigStorage('\t');
```



```
rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
[... 12 lines ...]
2026-09-11 20:23:42,535 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: GROUP_BY,FILTER
[... 48 lines ...]
2026-09-11 20:23:45,862 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1789136053647_0004
[... 2 lines ...]
2026-09-11 20:23:45,877 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 0% complete
[... 39 lines ...]
2026-09-11 20:24:10,090 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1789136053647_0005
[... 11 lines ...]
2026-09-11 20:24:35,436 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 100% complete
[... 2 lines ...]
HadoopVersion PigVersion UserId StartedAt FinishedAt Features
3.4.1 0.18.0 rii 2026-09-11 20:23:42 2026-09-11 20:24:35 GROUP_BY,FILTER
[... 1 line ...]
Success!
[... 2 lines ...]
JobId Maps Reduces MaxMapTime MinMapTime AvgMapTime MedianMapTime MaxReduceTime MinReduceTime AvgReduceTime MedianReduceTime Alias Feature Outputs
job_1789136053647_0004 1 1 3 3 3 3 1 1 1 cnt_for_ratings,games,grp_for_ratings,valid_games GROUP_BY,COMBINER
job_1789136053647_0005 1 1 5 5 5 5 4 4 4 grp_all,srt,top5_for_ratings GROUP_BY /steam/pig/Top5Ratings,
[... 2 lines ...]
Successfully read 50873 records (4864154 bytes) from: "/steam/games.csv"
[... 2 lines ...]
Successfully stored 5 records (89 bytes) in: "/steam/pig/Top5Ratings"
[... 27 lines ...]
2026-09-11 20:24:35,593 [main] INFO org.apache.pig.Main - Pig script completed in 54 seconds and 81 milliseconds (54081 ms)
rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop$
```

Figure 13: Execution: `top5Ratings.pig` — two jobs, 50,873 records read, 5 stored, in 54 seconds.

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ hdfs dfs -cat /steam/pig/Top5Ratings/part-+
2026-09-11 20:24:36,177 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Positive      13502
Very Positive 13139
Mixed         12157
Mostly Positive 8738
Mostly Negative 1849
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 14: Output: top 5 rating groups — the same five counts as the largest bins in Section 5.2.

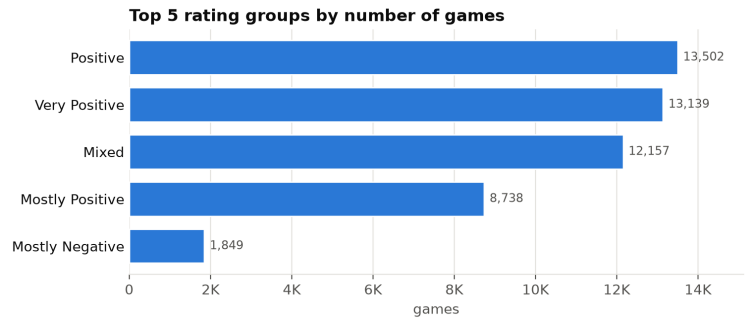


Figure 15: Visualization: top 5 rating groups by number of games.

6.2 Top 10 Most Reviewed Games

Returns the ten games with the most reviews on Steam (the `user_reviews` column). Counter-Strike: Global Offensive has 7,494,460 — more than three times the next game, PUBG: BATTLEGROUNDS (2,217,226). This column is Steam's own store-wide total, which is why the order differs from MapReduce job 1: that job counts only the reviews in `recommendations.csv`, where Team Fortress 2 comes first.

Script

```

1 -- Pig-2 Top 10 most reviewed games
2 games = LOAD '/steam/games.csv' USING PigStorage(',') AS
3   (app_id:int, title:chararray, date_release:chararray, win:chararray, mac:chararray,
4     linux:chararray, rating:chararray, positive_ratio:int, user_reviews:int,
5     price_final:double, price_original:double, discount:double, steam_deck:chararray);
6 valid_games = FILTER games BY app_id IS NOT NULL; -- drops the header row
7
8 grp_all = GROUP valid_games ALL;
9 top10 = FOREACH grp_all {
10   srt = ORDER valid_games BY user_reviews DESC;
11   lim = LIMIT srt 10;
12   GENERATE FLATTEN(lim);
13 };
14 final_top10_reviewed = FOREACH top10 GENERATE app_id, title, rating, positive_ratio, user_reviews;
15 STORE final_top10_reviewed INTO '/steam/pig/Top10Reviewed' USING PigStorage('\t');

```

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ pig -x mapreduce -f PigAnalysis/scripts/top10Reviewed.pig
[... 12 lines ...]
2026-09-11 20:24:39,655 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: GROUP_BY,FILTER
[... 45 lines ...]
2026-09-11 20:24:42,727 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapreduce.Launcher - HadoopJobId: job_1789136053647_0006
[... 2 lines ...]
2026-09-11 20:24:42,742 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapreduce.Launcher - 0% complete
[... 10 lines ...]
2026-09-11 20:25:25,292 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapreduce.Launcher - 100% complete
[... 2 lines ...]
HadoopVersion  PigVersion  UserId  StartedAt  FinishedAt  Features
3.4.1  0.18.0  rii  2026-09-11 20:24:40  2026-09-11 20:25:25  GROUP_BY,FILTER
[... 1 line ...]
Success!
[... 2 lines ...]
JobId  Maps  Reduces  MaxMapTime  MinMapTime  AvgMapTime  MedianMapTime  MaxReduceTime  MinReduceTime  AvgReduceTime  MedianReduceTime  Alias  Feature Outputs
job_1789136053647_0006  1  3  3  3  3  15  15  15  15  final_top10_reviewed,games,grp_all,srt,top10,valid_games  GROUP_BY  /
steam/pig/Top10Reviewed,
[... 2 lines ...]
Successfully read 50873 records (4864154 bytes) from: "/steam/games.csv"
[... 2 lines ...]
Successfully stored 10 records (486 bytes) in: "/steam/pig/Top10Reviewed"
[... 20 lines ...]
2026-09-11 20:25:25,672 [main] INFO org.apache.pig.Main - Pig script completed in 47 seconds and 736 milliseconds (47736 ms)
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 16: Execution: `top10Reviewed.pig` — one MapReduce job, 10 records stored in 48 seconds.

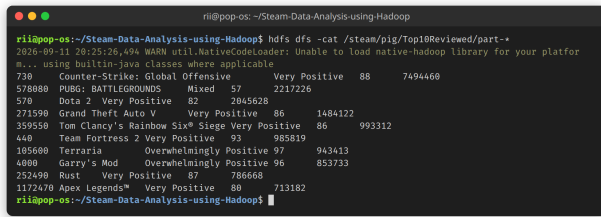


Figure 17: Output: top 10 most reviewed games (app_id, title, rating, positive ratio, user reviews).

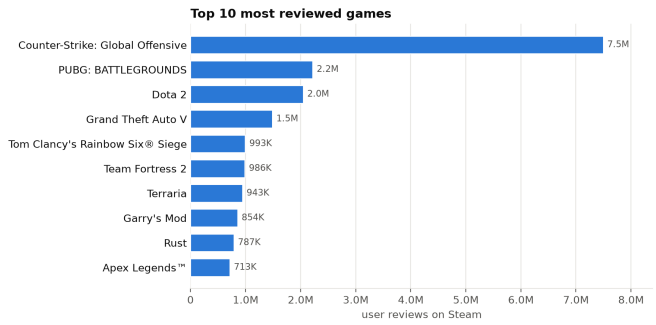


Figure 18: Visualization: top 10 most reviewed games.

6.3 Top 10 Most Reviewed Games in each Rating Group

Returns the ten most reviewed games inside each of the nine rating groups. After GROUP BY rating, the nested FOREACH block runs once per group, sorts that group's bag by user_reviews, keeps ten and FLATTENS them back into rows — one MapReduce job.

Script

```
1 -- Pig-3 Top 10 most reviewed games inside each rating group
2 games = LOAD '/steam/games.csv' USING PigStorage(',') AS
3   (app_id:int, title:chararray, date_release:chararray, win:chararray, mac:chararray,
4     linux:chararray, rating:chararray, positive_ratio:int, user_reviews:int,
5     price_final:double, price_original:double, discount:double, steam_deck:chararray);
6 valid_games = FILTER games BY app_id IS NOT NULL; -- drops the header row
7
8 grp_for_ratings = GROUP valid_games BY rating;
9 top10_reviewed_ratings = FOREACH grp_for_ratings {
10   sorted = ORDER valid_games BY user_reviews DESC;
11   top10 = LIMIT sorted 10;
12   GENERATE FLATTEN(top10);
13 };
14 top10_reviewed_by_rating = FOREACH top10_reviewed_ratings GENERATE rating, app_id, title, user_reviews;
15 STORE top10_reviewed_by_rating INTO '/steam/pig/Top10ReviewedByRating' USING PigStorage('\t');
```

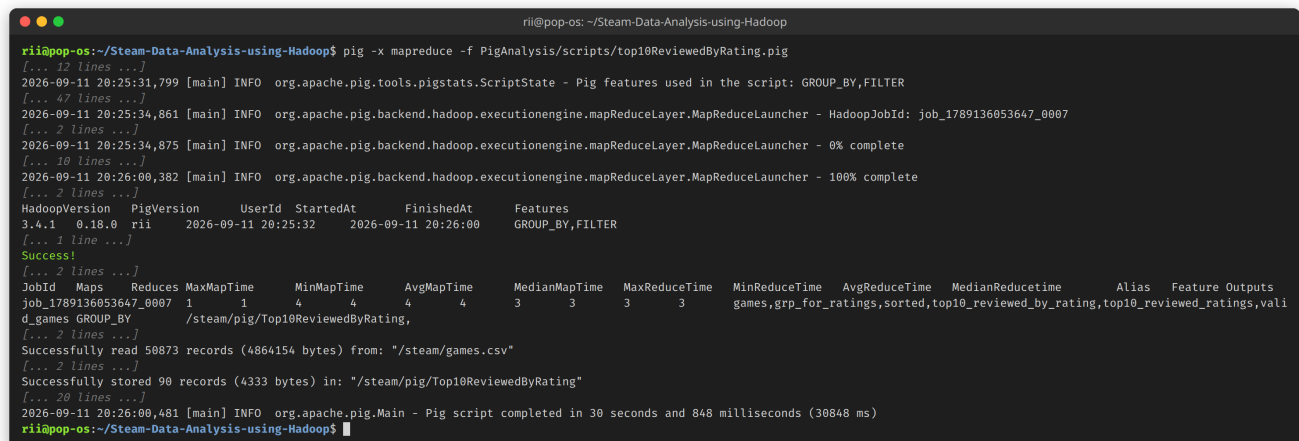


Figure 19: Execution: top10ReviewedByRating.pig — 90 records stored (nine groups of ten), in 31 seconds.

```

rli@pop-os: ~/Steam-Data-Analysis-using-Hadoop$ hdfs dfs -cat /steam/pig/Top10ReviewedByRating/part-1
2026-09-11 20:26:01,175 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Mixed 578880 PUBG: BATTLEGROUNDS 2217226
Mixed 1938898 Call of Duty® 429286
Mixed 433850 Z1 Battle Royale 287328
Mixed 1468810 鬼泣/Devil May Cry: Tales of Immortal 45
Mixed 1517298 Battlefield™ 2042 153151
Mixed 227940 Heroes & Generals 141919
Mixed 1811260 EA SPORTS™ FIFA 23 113742
Mixed 364390 FOR HONOR™ 82277
Mixed 291480 Warface 88549
Mixed 397980 Business Tour - Board Game with Online Multiplayer 65424
Negative 364710 Wonky Pigeon! 47
Negative 273880 Forestry 2017 - The Simulation 47
Negative 345710 Damned Nation Reborn 45
Negative 1829586 HITMAN 3 - Seven Deadly Sins Act 7: Wrath 44
Negative 318880 Basketball Pro Management 2015 44
Negative 218490 Fray: Reloaded Edition 42
Negative 925670 Dungeon Kitty 42
Negative 288380 Uprising4x: The Silent Shadows 42
Negative 519180 Streamer Simulator 42
Negative 315330 All Guns On Deck 42
Positive 591760 HyperBrawl Tournament 49
Positive 1748750 Silly Mouthed - 沉默/口无遮拦 49
Positive 1947750 Queen's Wish 2: The Tormentor 49
Positive 1727930 Gorge 49
Positive 1260130 Banana Hell 49
Positive 1282770 kirakira stars idol project AI 49
Positive 769680 武神联盟 49
Positive 1714190 Breakout: Recharged 49
Positive 2167451 Monster Hunter Rise - 'Ran Page' Hunter layered armor set 49
[... 61 more lines ...]
rli@pop-os: ~/Steam-Data-Analysis-using-Hadoop$

```

Figure 20: Output: the first 30 of the 90 rows (full output in Pig Results/).

Top 10 most reviewed games in each rating group

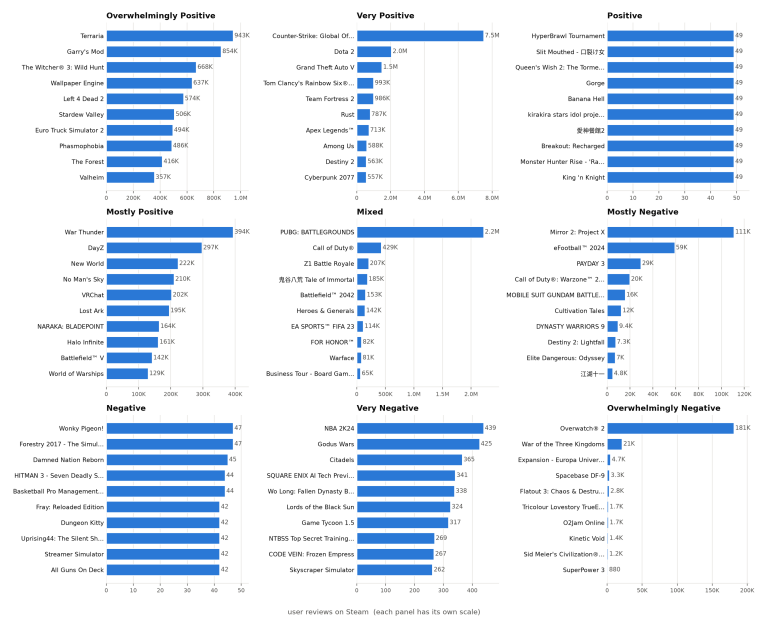


Figure 21: Visualization: top 10 most reviewed games in each rating group (each panel has its own scale).

Two things stand out. The “Positive” and “Negative” groups never go above 49 reviews: Steam only gives those plain labels to games with 10–49 reviews, and a game that collects more moves up to “Very” or “Overwhelmingly”. And the most reviewed game in the Overwhelmingly Negative group is Overwatch® 2, with 181,198 reviews — a very large game can sit in the worst group.

6.4 Top 10 Games by Total Hours Played

Returns the ten games whose reviewers played them for the most hours in total. This script reads the full 2 GB review table: it adds up the hours per game with GROUP BY app_id and SUM, then brings in each game’s title and rating with a **replicated join** — games.csv is small enough to be copied to every mapper, so the join happens on the map side without a second shuffle. The hours stay numeric for the sort and are formatted with SPRINTF only for output. Pig runs it as a chain of three MapReduce jobs: a map-only job that prepares games.csv for the replicated join, one with 15 maps and 3 reducers that groups the 41 million reviews and joins, and a final ranking job.

Script

```

1 -- Pig-4 Top 10 games by total hours played by their reviewers
2 recs = LOAD '/steam/recommendations.csv' USING PigStorage(',') AS
3     (app_id:int, helpful:int, funny:int, date:chararray, is_recommended:chararray,
4      hours:double, user_id:int, review_id:int);
5 valid_rec = FILTER recs BY app_id IS NOT NULL;    -- drops the header row
6
7 grp_for_games = GROUP valid_rec BY app_id;
8 played = FOREACH grp_for_games GENERATE group AS app_id,
9     COUNT(valid_rec) AS reviews,
10    SUM(valid_rec.hours) AS hours;
11
12 games = LOAD '/steam/games.csv' USING PigStorage(',') AS
13     (app_id:int, title:chararray, date_release:chararray, win:chararray, mac:chararray,
14     linux:chararray, rating:chararray, positive_ratio:int, user_reviews:int,
15     price_final:double, price_original:double, discount:double, steam_deck:chararray);
16 game_info = FOREACH (FILTER games BY app_id IS NOT NULL) GENERATE app_id, title, rating;
17
18 -- games.csv is small, so it is copied to every mapper (map-side replicated join)
19 joined = JOIN played BY app_id, game_info BY app_id USING 'replicated';
20 result = FOREACH joined GENERATE played::app_id AS app_id, game_info::title AS title,
21     game_info::rating AS rating, played::reviews AS reviews,
22     played::hours AS hours;
23
24 grp_all = GROUP result ALL;
25 top10 = FOREACH grp_all {
26     srt = ORDER result BY hours DESC;
27     lim = LIMIT srt 10;
28     GENERATE FLATTEN(lim);
29 };
30 -- hours stay numeric for the sort and are formatted only for output
31 final_top10_played = FOREACH top10 GENERATE app_id, title, rating, reviews,
32     SPRINTF('%1f', hours) AS hours;
33 STORE final_top10_played INTO '/steam/pig/Top10Played' USING PigStorage('\t');

```

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ pig -x mapreduce -f PigAnalysis/scripts/top10Played.pig
[... 12 lines ...]
2026-09-11 20:38:17,976 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: REPLICATED_JOIN, GROUP_BY, FILTER
[... 49 lines ...]
2026-09-11 20:38:20,520 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1789136053647_0014
[... 2 lines ...]
2026-09-11 20:38:20,536 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 0% complete
[... 42 lines ...]
2026-09-11 20:38:44,186 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1789136053647_0015
[... 48 lines ...]
2026-09-11 20:42:29,885 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1789136053647_0016
[... 11 lines ...]
2026-09-11 20:42:50,439 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 100% complete
[... 2 lines ...]
HadoopVersion PigVersion Userid StartedAt FinishedAt Features
3.4.1 0.18.0 rii 2026-09-11 20:38:18 2026-09-11 20:42:50 REPLICATED_JOIN, GROUP_BY, FILTER
[... 1 line ...]
Success!
[... 2 lines ...]
JobId Maps Reduces MaxMapTime MinMapTime AvgMapTime MedianMapTime MaxReduceTime MinReduceTime AvgReduceTime MedianReduceTime Alias Feature Outputs
job_1789136053647_0014 1 0 6 6 6 0 0 0 0 1-2, game_info, games MAP_ONLY REPLICATED_JOIN, GROUP_BY,
job_1789136053647_0015 15 3 66 36 52 52 140 10 65 47 grp_for_games, joined, played, recs, result, valid_rec COMBINER
job_1789136053647_0016 1 1 3 3 3 3 3 3 3 3 final_top10_played, grp_all, srt, top10 GROUP_BY /steam/pig/Top10Played,
[... 2 lines ...]
Successfully read 50873 records (4864154 bytes) from: "/steam/games.csv"
Successfully read 41154795 records (2023286021 bytes) from: "/steam/recommendations.csv"
[... 2 lines ...]
Successfully stored 10 records (555 bytes) in: "/steam/pig/Top10Played"
[... 34 lines ...]
2026-09-11 20:42:51,889 [main] INFO org.apache.pig.Main - Pig script completed in 4 minutes, 34 seconds and 925 milliseconds (274925 ms)
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 22: Execution: top10Played.pig — three jobs, 41,154,795 review lines and 50,873 game lines read, 10 records stored, in 4 minutes 35 seconds.

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ hdfs dfs -cat /steam/pig/Top10Played/part-
2026-09-11 20:42:52,686 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform
m... using builtin-java classes where applicable
440 Team Fortress 2 Very Positive 319492 181809977.6
730 Counter-Strike: Global Offensive Very Positive 219737 94260307.3
578 Dota 2 Very Positive 216914 93072785.4
252498 Rust Very Positive 270684 92944004.6
359550 Tom Clancy's Rainbow Six® Siege Very Positive 189683 68445823.3
1085060 Destiny 2 Very Positive 183527 62504172.8
394360 Hearts of Iron IV Very Positive 143187 53086631.1
346110 ARK: Survival Evolved Very Positive 159736 46720359.4
230410 Warframe Very Positive 127812 46450175.3
271590 Grand Theft Auto V Very Positive 163909 46170394.5
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 23: Output: top 10 games by total hours played (app_id, title, rating, reviews, total hours).

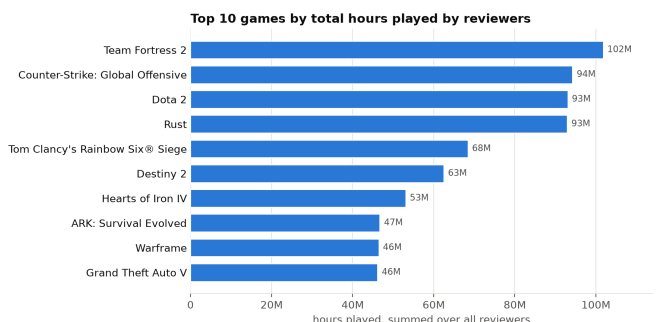


Figure 24: Visualization: top 10 games by total hours played.

The list is identical to the top ten of the MapReduce join (Section 5.3) — same games, same order, same review counts, same hours to one decimal place. The hand-written join took 1 minute 15 seconds and the Pig script 4 minutes 35 seconds, mostly because Pig runs three jobs one after another while the Java version does everything in one job and shrinks its shuffle with in-mapper combining.

6.5 Top 10 Games by Total Hours Played in each Rating Group

Returns the ten most played games inside each rating group. The same first half as the previous script — sum the hours per game and replicated-join the titles and ratings — followed by the nested sort of Section 6.3, grouped by rating.

Script

```
1 -- Pig-5 Top 10 games by total hours played inside each rating group
2 recs = LOAD '/steam/recommendations.csv' USING PigStorage(',') AS
3     (app_id:int, helpful:int, funny:int, date:chararray, is_recommended:chararray,
4      hours:double, user_id:int, review_id:int);
5 valid_rec = FILTER recs BY app_id IS NOT NULL;    -- drops the header row
6
7 grp_for_games = GROUP valid_rec BY app_id;
8 played = FOREACH grp_for_games GENERATE group AS app_id,
9     COUNT(valid_rec) AS reviews,
10    SUM(valid_rec.hours) AS hours;
11
12 games = LOAD '/steam/games.csv' USING PigStorage(',') AS
13     (app_id:int, title:chararray, date_release:chararray, win:chararray, mac:chararray,
14     linux:chararray, rating:chararray, positive_ratio:int, user_reviews:int,
15     price_original:double, price_discount:double, steam_deck:chararray);
16 game_info = FOREACH (FILTER games BY app_id IS NOT NULL) GENERATE app_id, title, rating;
17
18 joined = JOIN played BY app_id, game_info BY app_id USING 'replicated';
19 result = FOREACH joined GENERATE game_info::rating AS rating, played::app_id AS app_id,
20     game_info::title AS title, played::reviews AS reviews,
21     played::hours AS hours;
22
23 grp_for_ratings = GROUP result BY rating;
24 top10_played_ratings = FOREACH grp_for_ratings {
25     sorted = ORDER result BY hours DESC;
26     top10 = LIMIT sorted 10;
27     GENERATE FLATTEN(top10);
28 };
29 -- hours stay numeric for the sort and are formatted only for output
30 final_top10_played_by_rating = FOREACH top10_played_ratings GENERATE rating, app_id, title, reviews,
31     SPRINTF('%0.1f', hours) AS hours;
32 STORE final_top10_played_by_rating INTO '/steam/pig/Top10PlayedByRating' USING PigStorage('\t');
```

```

rii@pop-os: ~/Steam-Data-Analysis-using-Hadoop
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$ pig -x mapreduce -f PigAnalysis/scripts/top10PlayedByRating.pig
[... 12 lines ...]
2026-09-11 20:42:57,087 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: REPLICATED_JOIN, GROUP_BY, FILTER
[... 49 lines ...]
2026-09-11 20:43:00,707 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1789136053647_0017
[... 2 lines ...]
2026-09-11 20:43:00,720 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 0% complete
[... 42 lines ...]
2026-09-11 20:43:17,577 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1789136053647_0018
[... 48 lines ...]
2026-09-11 20:46:36,757 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1789136053647_0019
[... 11 lines ...]
2026-09-11 20:46:57,352 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 100% complete
[... 2 lines ...]
HadoopVersion PigVersion UserId StartedAt FinishedAt Features
3.4.1 0.18.0 rii 2026-09-11 20:42:57 2026-09-11 20:46:57 REPLICATED_JOIN, GROUP_BY, FILTER
[... 1 line ...]
Success!
[... 2 lines ...]
JobId Maps Reduces MaxMapTime MinMapTime AvgMapTime MedianMapTime MaxReduceTime MinReduceTime AvgReduceTime MedianReduceTime Alias Feature Outputs
job_1789136053647_0017 1 0 2 2 2 2 0 0 0 0 1-2,game_info,games MAP_ONLY
job_1789136053647_0018 15 3 58 31 47 51 80 9 53 71 grp_for_games,joined,played,recs,result,valid_rec REPLICATED_JOIN, GROUP_BY, COMBINER
job_1789136053647_0019 1 1 2 2 2 2 2 2 2 2 final_top10_played_by_rating,grp_for_ratings,sorted,top10_played_ratings G
ROUP_BY /steam/pig/Top10PlayedByRating,
[... 2 lines ...]
Successfully read 50873 records (4864154 bytes) from: "/steam/games.csv"
Successfully read 41154795 records (2023286021 bytes) from: "/steam/recommendations.csv"
[... 2 lines ...]
Successfully stored 87 records (4687 bytes) in: "/steam/pig/Top10PlayedByRating"
[... 34 lines ...]
2026-09-11 20:46:58,390 [main] INFO org.apache.pig.Main - Pig script completed in 4 minutes, 3 seconds and 280 milliseconds (243280 ms)
rii@pop-os:~/Steam-Data-Analysis-using-Hadoop$

```

Figure 25: Execution: top10PlayedByRating.pig — 87 records stored, in 4 minutes 3 seconds.

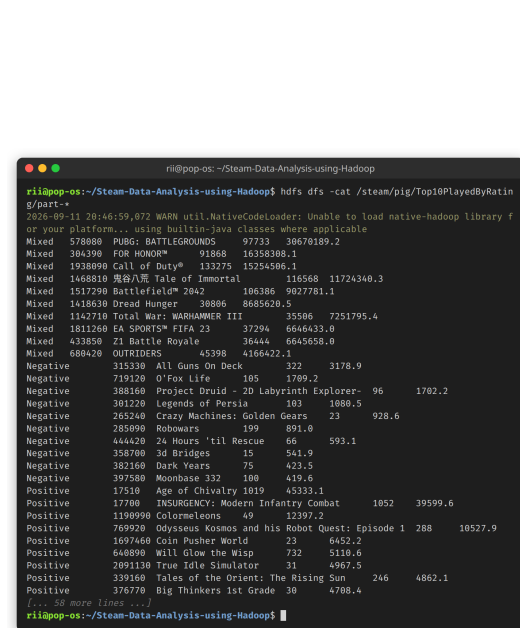


Figure 26: Output: the first 30 of the 87 rows (full output in Pig Results/).

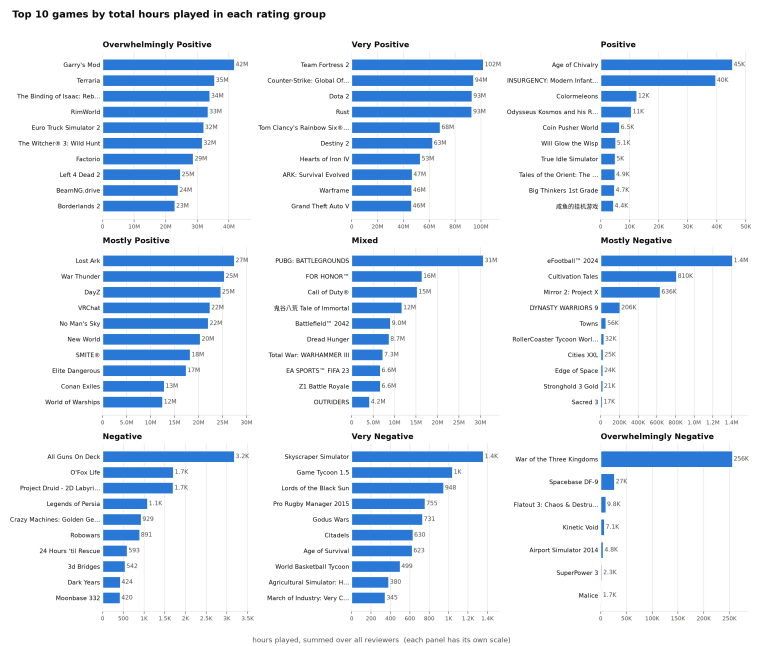


Figure 27: Visualization: top 10 games by total hours played in each rating group (each panel has its own scale).

The script stored 87 rows, not 90: the Overwhelmingly Negative group has 14 games, but only 7 of them have any review in recommendations.csv, so that group returns seven.

7. Monitoring the Jobs

Hadoop's web interfaces were kept open while the jobs ran. The ResourceManager lists 19 applications, all SUCCEEDED: the three MapReduce jobs and the MapReduce jobs Pig generated for the five scripts. Scripts 4 and 5 appear twice — they were run once, then again after changing only how the hours are printed (from Pig's default scientific notation to one decimal place); the second runs are the ones in this report.

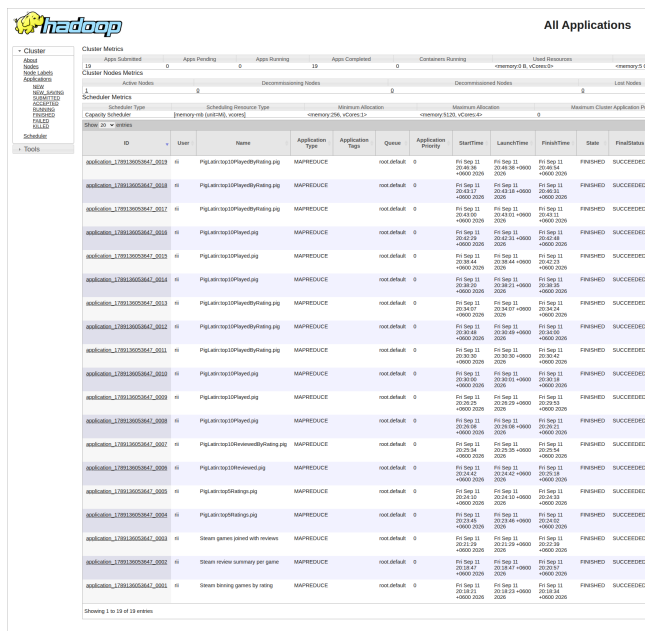


Figure 28: YARN ResourceManager (localhost:8088): all 19 applications SUCCEEDED.

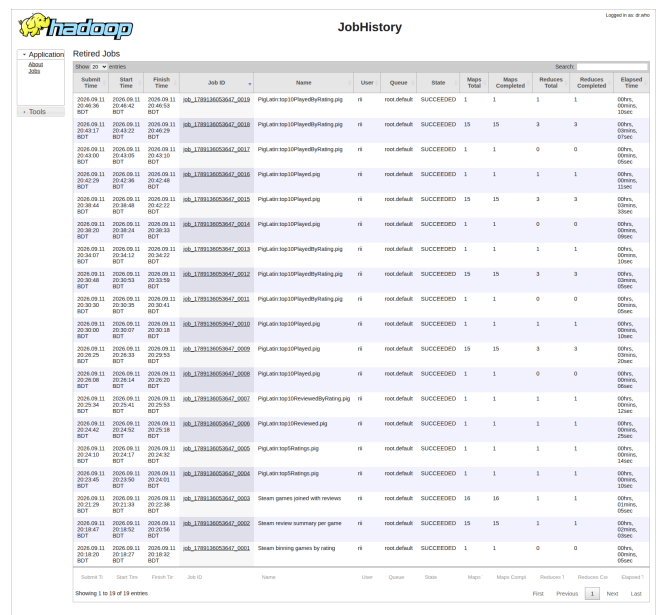


Figure 29: MapReduce JobHistory (localhost:19888) with the map and reduce task counts of every job.

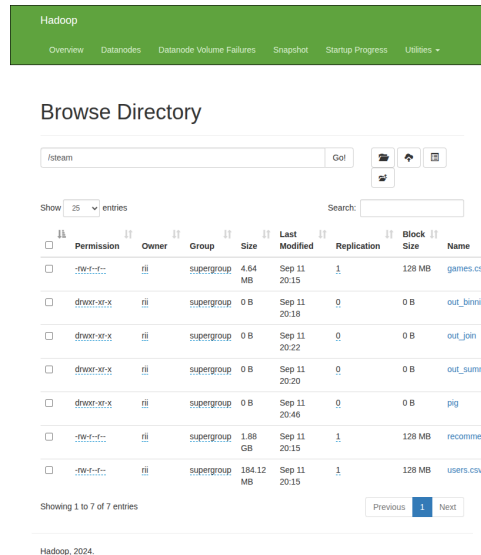


Figure 30: The /steam directory in the HDFS browser: the three input files and the output directories of every job.

8. Verification

Different jobs reach several of the same numbers by different routes, so the results can be checked against each other and against the raw file. All checks passed. The bin sizes were also counted straight from the CSV before any job ran, and the Pig-versus-MapReduce top ten was compared with diff.

Check	Expected	Produced	
Reviews read by the summary job = rows of recommendations.csv	41,154,794	41,154,794	pass
Reviews summed over the summary output = over the join output	41,154,794	41,154,794	pass
Games in the summary output = games matched by the join	37,610	37,610	pass
Joined games + games with no reviews = rows of games.csv	50,872	37,610 + 13,262	pass
Nine bins of the binning job add up to all games	50,872	50,872	pass
Bin sizes = counts computed directly from games.csv	9 values	all 9 equal	pass
Pig top 5 rating groups = the five largest bins	5 values	all 5 equal	pass
Pig top 10 by hours = MapReduce join top 10 (ids, reviews, hours)	10 rows	identical	pass
Pig-5 rows = 8 groups × 10 + 7 reviewed Overwhelmingly Negative games	87	87	pass

Table 7: Cross-checks between jobs.

9. What the Data Says

Steam reviews are mostly positive. 85.8% of the 41 million reviews recommend the game, and 71.7% of all games carry one of the four positive labels. Only 2,226 games (4.4%) carry a negative one, and just 14 are Overwhelmingly Negative.

Attention is concentrated in a few games. Of the 50,872 games, 37,610 appear in the review data at all, and half of those have 39 reviews or fewer. The ten most reviewed games — 0.03% of the reviewed games — hold 5.4% of all reviews.

The most played games are long-running multiplayer titles. Team Fortress 2, Counter-Strike: Global Offensive, Dota 2 and Rust each passed 90 million hours of play among their reviewers, and the average Counter-Strike or Dota 2 reviewer had played about 429 hours. Across all reviews the players had logged 4.14 billion hours.

Popular does not mean well rated. PUBG: BATTLEGROUNDS is the second most reviewed game on Steam but is rated Mixed, with 57% positive reviews; Overwatch® 2 has 181,198 reviews and is Overwhelmingly Negative. At the other end, Wallpaper Engine is 98.2% positive in the review data.

A rating label is partly a volume label. The Positive and Negative groups top out at 49 reviews, because Steam reserves the “Very” and “Overwhelmingly” labels for games with more reviews. Grouping by rating therefore mixes how good the reviews are with how many there are — worth keeping in mind before comparing the groups directly.

10. Conclusion

A 2.2 GB, three-table Steam dataset was loaded into HDFS and analysed on a single-node Hadoop 3.4.1 cluster with three Java MapReduce programs and five Pig Latin scripts, all of which completed successfully. The summarization job shows how a custom Writable and a combiner shrink 41 million map outputs to under a hundred thousand before the shuffle; the binning job shows a map-only job writing many outputs at once; and the reduce-side join shows two differently shaped inputs meeting in one reducer, with in-mapper combining making the join over 41 million rows finish in just over a minute. Pig answered the ranking questions in 15 to 33 lines per script, including a replicated join over the full review table, but took roughly three to four times as long as the hand-tuned Java join for the same answer. The two tools agreed on every result they share, which is the strongest evidence that both are correct.

References

- [1] A. Kozyriev, *Game Recommendations on Steam*, Kaggle dataset. <https://www.kaggle.com/datasets/antonkozyriev/game-recommendations-on-steam>
- [2] S. Prabhu, *YouTube Data Analysis using Hadoop* (project structure followed). <https://github.com/SiddhiPrabhu1995/YouTube-Data-Analysis-using-Hadoop>
- [3] D. Miner and A. Shook, *MapReduce Design Patterns*, O'Reilly, 2012.
- [4] Apache Hadoop 3.4.1 documentation. <https://hadoop.apache.org/docs/r3.4.1/>
- [5] Apache Pig 0.18.0 documentation. <https://pig.apache.org/docs/latest/>